Holant lead, step (a): the entry-access GGM lemma (H1) and the transfer-matrix spectrum

Lead: LEAD-EXO-holant-decomposition-count-network, next_action step (a).

Status. This is a derivation, with numerical checks of the derivation at toy primes (p < 400; every script runs in seconds). It is not a measurement and carries no cost claim. It changes no research status by itself. An independent red-team review (REDTEAM.md, Opus) found faults in the first draft. This version adopts its restatements; the review is kept verbatim.

Summary

item	lead text (2026-11-01)	finding
H1	"An algorithm that only reads entries of T is simulable in the generic group model with O(1) group operations per entry."	**True only under a reading that excludes every basis change; false under the reading the lead needs.** See the two readings below.
Consequence of (a)	"only bases that use the algebraic form of S_3 remain in scope"	**Does not follow.** The GGM excludes no basis change on the F_p alphabet. Step (b) is the test, unchanged.
Transfer matrix, odd #E	eigenfunctions chi_j + chi_{-j} "up to multiplicity conventions"	**Corrected.** M is the symmetrised Cayley quotient **with the O vertex deleted**. Its eigenvalues are the zeros of the secular equation sum_j 1/(lambda_j - mu) = 0, plus degenerate lambda_j with multiplicity reduced by one. Its eigenvectors are Green functions G_mu(P) = sum_j cos(2 pi j log P / n)/(lambda_j - mu): still explicit functions of log P, but not single characters.
Transfer matrix, even #E	(same)	**Different again.** The 2-torsion rows carry a factor 2 relative to the quotient, and interlacing fails (3/3 curves).
j = 0, automorphism-stable FB	(same)	Two thirds of M's spectrum is exactly character sums (11/11 curves), because the lambda_j are degenerate. This is the secp256k1 case.
Alphabet F_p	a single network	Splits into an E(F_p) block and a quadratic-twist block. With FB on E and odd #E, the cross mass and the twist-block mass are 0 at every toy prime tested.

H1: two readings

Model (Shoup). Group elements are seen as sigma(P), for a uniformly random injection sigma. An oracle answers sigma(P), sigma(Q) -> sigma(P - Q), and the public data is (p, a, b, n, FB). Shoup's bound counts **oracle queries only**: success probability is O(q^2/n') for q queries, where n' is the largest prime dividing n. Local computation is free.

Lemma 1 (the bulk is public). T, its FB-leaf contractions and M depend only on (p, a, b, FB). Building or transforming them uses 0 oracle queries. This is a query count, not a time bound: building M at p = 307 costs about 9·10^4 S_3 evaluations, which the note leaves unpriced.

Reading A: label-indexed entries. The algorithm may ask for T[x(R1), x(R2), x(R3)] on group labels sigma(R_i). This is simulable with at most 4 group queries per entry (one subtraction, one addition and two equality tests). The red-team checked 287,496 triples with 0 mismatches (rt_h1_paths.py). Under this reading H1 is **true** and Shoup's bound applies. The class, however, can only test incidences of labelled points; it **cannot apply any basis change on the F_p alphabet**, because it never sees field elements. It therefore says nothing about the holographic question the lead asks.

Reading B: field-indexed entries with an instance boundary. The decomposition count of a target R is <e_{x(R)}, C_m>, with C_m public. A holographic algorithm transforms the F_p bond spaces, so it needs x(R) as a field element, which means an oracle X: sigma(R) -> x(R).

Lemma 3 (restated). On every curve, X cannot be simulated with o(sqrt n') group queries per query. *Proof.* Two X-queries return x(P) and x(Q). Free local computation then gives log_P Q up to sign. Resolving the sign costs O(log n) group queries (form sigma([k]P) by double-and-add and compare it with sigma(Q)). If each X-query cost s group queries, the result would be a generic algorithm with 2s + O(log n) queries, so Shoup forces s = Omega(sqrt n'), and baby-step giant-step attains it. ∎ The first draft used anomalous curves and Smart's attack. That route is sound but unnecessary, it miscounted the sign step as one query, and it suggested wrongly that simulability is curve-specific.

What Lemma 3 does **not** say is whether x-access helps a *time-bounded* attacker on the lab's prime-order targets. No GGM argument can decide that, since the GGM charges no time to local computation. That question is index calculus itself.

The repository's "structured GGM" (KN-FIND-002, which makes coordinates public) gives no lower bound under either reading. In it simulability is vacuous, and Shoup's theorem does not apply.

Consequence for the lead

- H1 should be replaced by the two readings above. Reading A is closed at exponent 1/2 but excludes every basis change; reading B is outside the GGM.
- Step (a) removes **no** basis family from scope. Step (b), building T and M at p ≤ 2^10 and comparing flattening ranks after basis changes against a random-support null, is the discriminating test, unchanged.
- The sentence "a basis that diagonalises M encodes the discrete log" survives in its Green-function form. The eigenvectors are resolvent-weighted character sums of log P.
- H2 (Cai–Chen needs a fixed finite domain) is untouched.

Transfer-matrix checks

Odd #E, smallest-x FB with |FB| = floor(sqrt p), first a with prime #E at b = 1 (check_transfer.py, check_O.py):

p	n	interlaces	M = char sums	with O vertex = char sums (max err)	char residual without O	twist mass
61	67	yes	no	yes (3.6e-14)	0.656	0
101	103	yes	no	yes (8.0e-14)	0.629	0
163	157	yes	no	yes (1.5e-13)	0.556	0
211	223	yes	no	yes (1.8e-13)	0.503	0
307	313	yes	no	yes (3.2e-13)	0.468	0

The positive control was to restore the O vertex with coupling sqrt 2. With it the spectrum equals the character sums to within 4e-13, so the deleted vertex is the whole discrepancy.

The red-team's independent checks (rt_transfer.py, rt_edge.py, rt_secular.py) give these results, and all four of its outputs reproduced byte for byte on rerun:

- Odd n, 21 further configurations (other b, other FB): the identity M = B[1:,1:] is exact.
- Even n, 3 curves at p = 67 (n = 56, 66, 82): the mismatched entries all lie in 2-torsion rows, where the ratio to the quotient is exactly 2. Interlacing is violated (16, 7 and 4 violations).
- j = 0 (a = 0) with a zeta-stable FB, 11 curves: 2/3 of M's eigenvalues equal character sums exactly.
- The secular equation and Green-function eigenvectors hold at all 5 producer curves: relative secular residual ≤ 1.9e-10, eigenvector cosine ≥ 1 - 4e-16.

The correct pairing condition in the proof is this. For each x_F, at most one of the four equalities P' = ±(P ± F) holds iff 2P, 2P' and 2F are all ≠ O. That is why the identity is for odd n only.

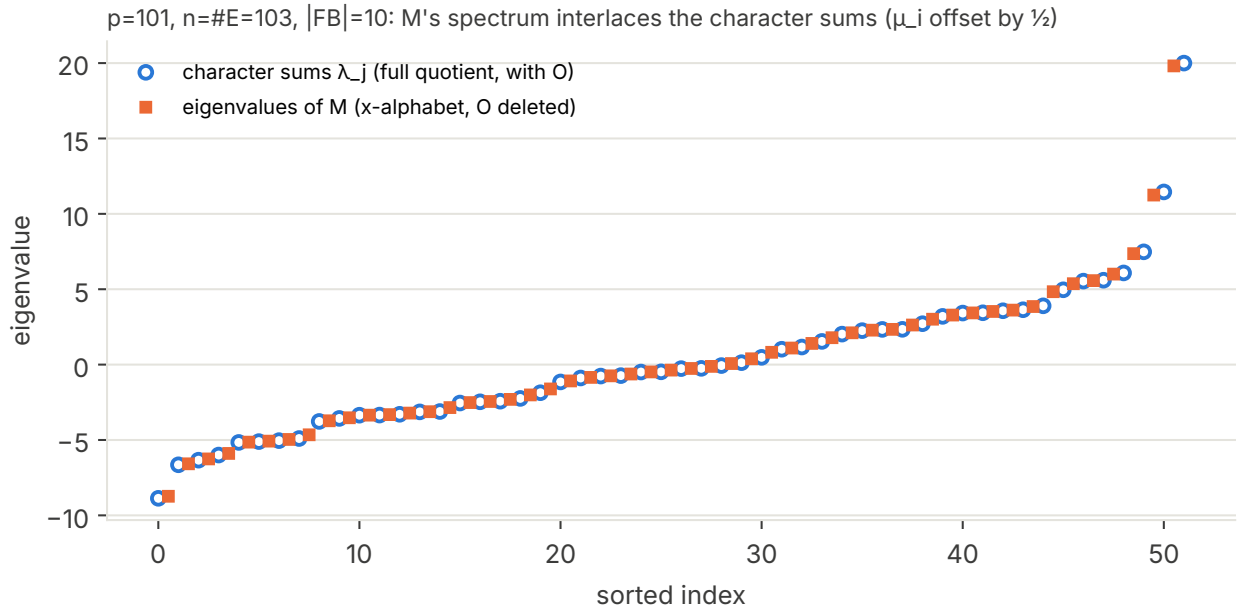

Figure 1: Spectrum of M against the character sums, p = 101

The augmented-oracle row (prior art, flagged, not changed)

LEAD-EXO-augmented-oracle-ggm (closed_obstructed) says that each named oracle, the x-oracle among them, is GGM-simulable, so Shoup closes its family at exponent 1/2. Lemma 3 contradicts this for the x-oracle. The contradiction is **not new here**. It is already in records the row cites or that supersede them:

- KN-FIND-194294 (cited by the row) classifies the x-oracle as NON-SIMULABLE (Tier 3).
- EV-GGM-002 supersedes EV-GGM-001 (cited by the row as "strong"). It states that the closure language of EV-GGM-001 is **not** carried forward.
- EV-ECDLP-048824, CC-3, raised the same contradiction for KN-FIND-002 and routed it to a separate review-breakthrough round.

On these records the row's own revisit_when condition, "a publicly computable oracle that is NOT GGM-simulable is exhibited", has fired. This note does not change the row. The routed round decides.

Reproduce

```
python3 check_transfer.py > transfer_spectrum.json
python3 check_O.py        > o_vertex_control.json
python3 figures.py        # spectrum-interlacing.svg
cd redteam && for s in rt_transfer rt_edge rt_secular rt_h1_paths; do python3 $s.py > $s.json; done
```

All scripts are pure Python plus NumPy (and matplotlib for the figure), and deterministic. build_pdf.sh rebuilds report.pdf.